

Hacking G Suite: The Power of Dark Apps Script Magic

By @IAmMandatory (Matthew Bryant)

Who am I?

- mandatory (Matthew Bryant)
- Leading the Red Team Effort at Snapchat
- @IAmMandatory
- Hacking write ups and research at <https://thehackerblog.com>

Context & Background

What exactly are we dealing with here?

Google Workspace (AKA G Suite)

- Suite of Google services for employees to work together online (Gmail, Docs/Sheets/Slides, Drive, GCP etc).
- Businesses can manage employees and set up powerful security and ACL policies.
- Many of these services are used individually by free Google users as well.
- >2 billion users!



Apps Script: Automate Google Services With JavaScript

- Serverless JavaScript apps which are hosted by Google and highly optimized for automating Google services.
- Seamless integration with Google's app registration and OAuth permission requesting system.
- Variety of triggers to start scripts: web request, document open, scheduled, etc.





Apps Script

Defcon Apps Script Example

Deploy



Files



Run



Debug

main



Execution log

Use legacy editor



Code.gs



Libraries



Services



Drive

Gmail

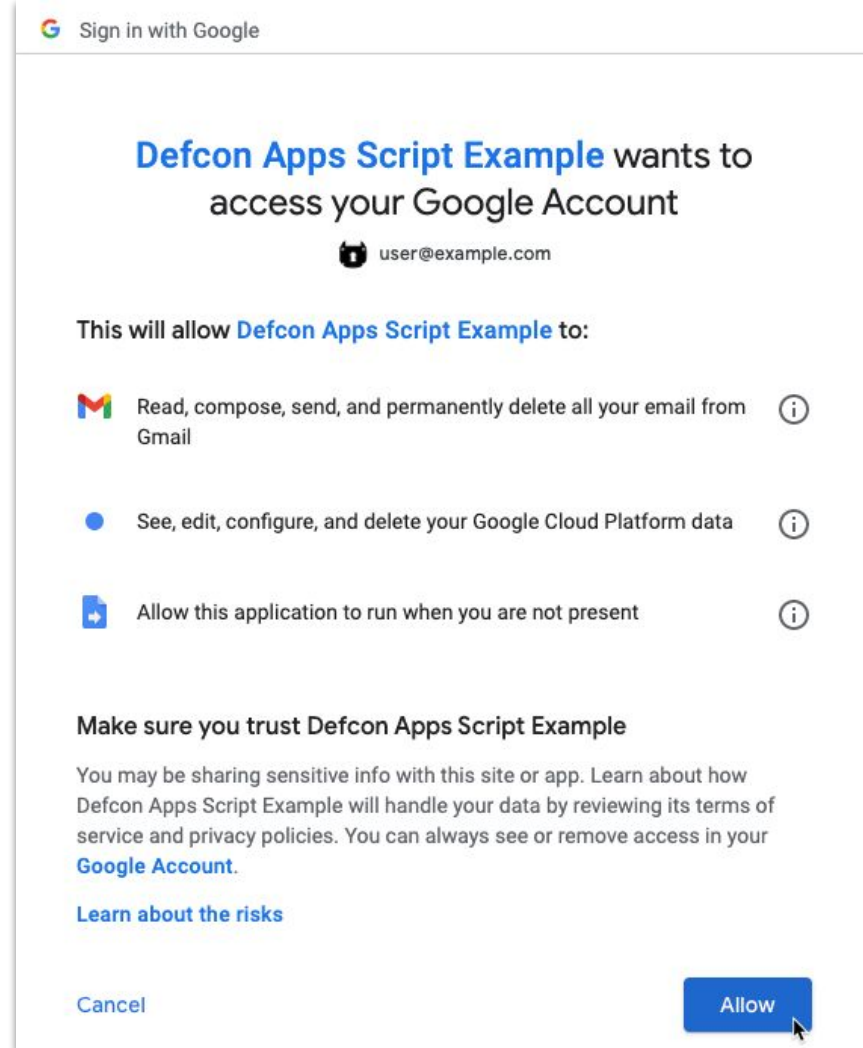
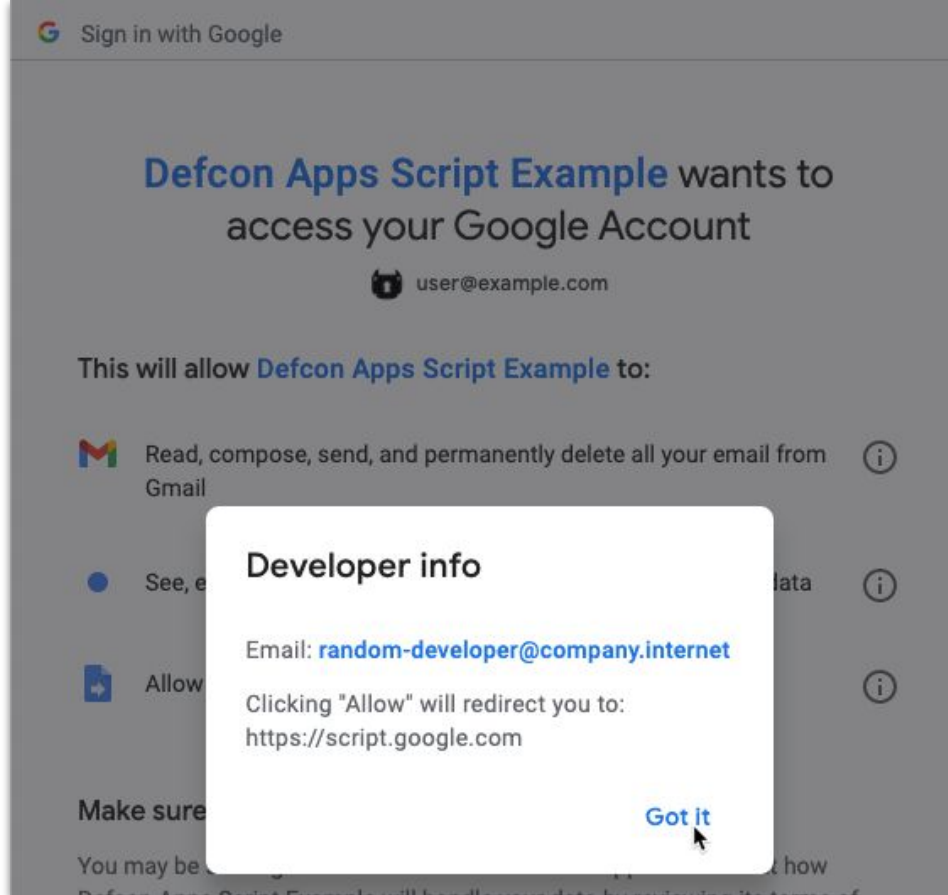
```
1 function main() {  
2   // List all the folders in Google Drive  
3   var folders = DriveApp.getFolders();  
4   while (folders.hasNext()) {  
5     var folder = folders.next();  
6     Logger.log(folder.getName());  
7   }  
8   DriveApp.f  
9 }  
10
```

- addFile (method) DriveApp.add...
- addFolder
- continueFileIterator
- continueFolderIterator
- createFile
- createFolder

Example of the Apps Script Editor

Google OAuth 2.0

- Allows third party apps to request access to resources owned by Google users.
- Permissions to resources are known as “scopes” and there are over 250 of them!
- Users are presented with a prompt describing roughly what access they’re granting which they can allow/reject.
- On approval you get tokens which you can use for the API(s)



Example OAuth Permission Prompts

Tying It All Together: Thinking Beyond the Machines

- When attacking G Suite, Apps Script is an attractive option for phishing as well as backdooring accounts.
- An Apps Script implant is outside the eyes of antivirus, endpoint detection tooling, and other on-device monitoring.
- Even if your victim wipes their laptop, your implant remains!

Tough Perimeters Require Clever Attacks

- We can utilize Apps Script to pierce even the most hardened environments.
- Companies with mandatory hardware U2F on logins, hardened Chromebooks, hardware attestation, third-party OAuth scope blocking, etc.
- To get around these measures we'll have to be a bit more clever than your average attacker...

Historical Precedent

*“There are years where we fuck around;
and there are hours where we find out”*

Joe Bernstein has shared a document on Google Docs with you



Inbox x



joe.bernstein@buzzfeed.com

🔒 to hhhhhhhhhhhhhh., bcc: zeynep ▾

Joe Bernstein has invited you to view the following document:

[Open in Docs](#)

Image from @zeynep Tweet: <https://bit.ly/3vDvy1z>



▼ Google Docs would like to:



Read, send, delete, and manage your email



Manage your contacts



By clicking Allow, you allow this app and Google to use your information in accordance with their respective [terms of service](#) and [privacy policies](#). You can change this and other [Account Permissions](#) at any time.

Deny

Allow

Image from Ars Technica article: <https://bit.ly/2SMGqgb>

A Modern-Day Super Worm

SOME USERS AUTHORIZE ACCESS

A percentage of users who receive the phishing email fall for it. They grant the worm access to their Gmail and contacts.



SPREAD

Worm sends out OAuth phishing email using victim's Gmail to their 1,000 most recently modified contacts.

USERS RECEIVE PHISHING EMAIL

More people receive the phishing email requesting full access to their Gmail and Contacts list.

The Impact

- The worm spread like wildfire and affected >1 million Google users (personal and enterprise alike).
- Google rapidly responded to the incident and was able to halt the spread and kill the apps in a few hours.
- Post-mortem analysis indicated the coding was amateur and only collected email addresses.
- Everything considered, this attack could've been **much** worse.

The Attack Components

- Multiple rotating apps and domains to prevent easy-blocking by Google
- IDN homograph attack in app name
- Social engineering & clever phishing scheme
- Self-propagation via old-school email spam

Post-Worm Changes & Existing Mitigations

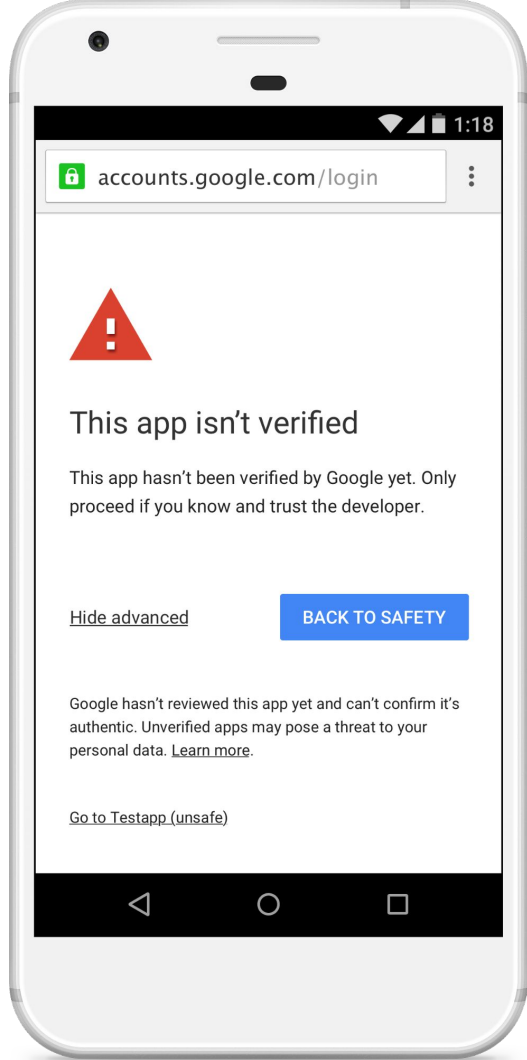
- ~2 months later Google introduced G Suite OAuth scope and client ID allowlisting.
- Google later introduced “Sensitive” and “Restricted” scopes which require heavy review.
- Introduction of the “Unverified App” warning prompt for smaller apps requiring the aforementioned scopes.
- Crackdown on all misleading OAuth apps.

Food for thought:

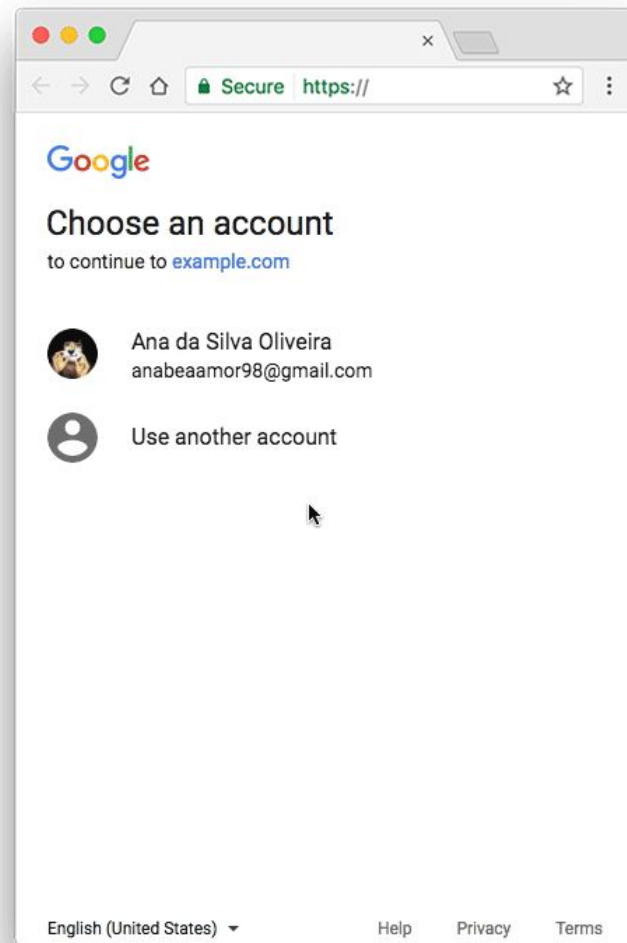
This attack utilized zero exploits or bugs, yet the impact was substantial.

Breaking New Ground

Bypassing the new restrictions to pierce the G Suite perimeter



The “Unverified App” prompt



What is a “sensitive” or “restricted” scope?

- Any API with potential to access private data: Gmail, BigQuery, GCP, Groups, Drive, Calendar, etc. (*Over ~120 total APIs*)
- For apps with <100 users, “Unverified App” prompt
- For apps with >100 users you need to undergo an intense manual review process.
- However, there *are* some exceptions...

For Apps Script, what causes the “Unverified App” prompt?

The application uses a “sensitive” OAuth scope *and...*

	Client is verified	Publisher is a Google Workspace account of customer A	Script is in a shared drive of customer A	Publisher is a Gmail account
User is a Google Workspace account of customer A	Normal auth flow	Normal auth flow	Normal auth flow	<i>Unverified auth flow</i>
User is a Google Workspace account <u>not</u> of customer A	Normal auth flow	<i>Unverified auth flow</i>	<i>Unverified auth flow</i>	<i>Unverified auth flow</i>
User is a Gmail account ¹	Normal auth flow	<i>Unverified auth flow</i>	<i>Unverified auth flow</i>	<i>Unverified auth flow</i>

¹Any Gmail account, including the account used to publish the app.

*If the publisher of the app
is in the same G Suite as
the user running it, then
there will be no
“Unverified App” prompt.*

Getting Clever With Apps Script

- Apps Script apps can be a standalone project or they can be bound to Google Docs/Sheets/Slides/Forms.
- This allows for manipulation of the document, customization of the UI, etc.
- Apps Script triggers will run for all users who view the doc with Editor permissions (they still have to accept OAuth prompt for scopes).



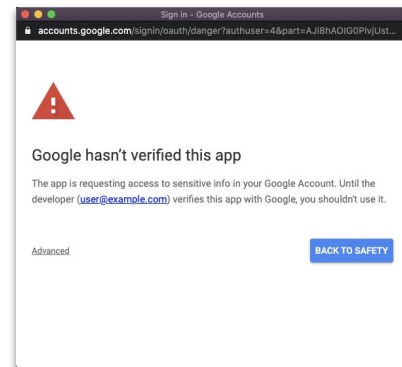
Attacker

Make victim "Editor"
on Doc and send link



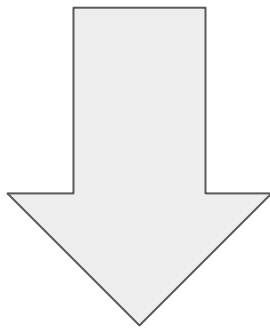
Victim
(In G Suite Org)

Victim activates trigger
and spawns prompt



Grant User "Editor" Attempt: **FAILED**

/d/1J1...xA/**edit**



/d/1J1...xA/**copy**





Google Sheets

Copy document

Would you like to make a copy of **[Confidential] Org-wide Comp & Promo Details?**

Make a copy



Example dialogue shown from copy URL
(Copied sheet is opened immediately upon clicking “Make a copy”)



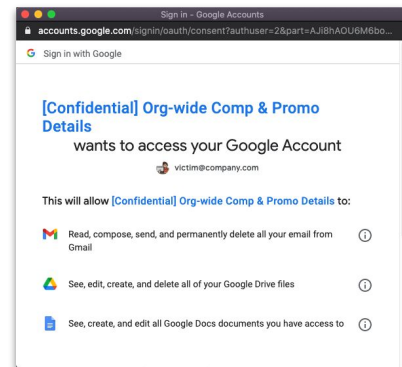
Attacker

Send victim the copy
link to document

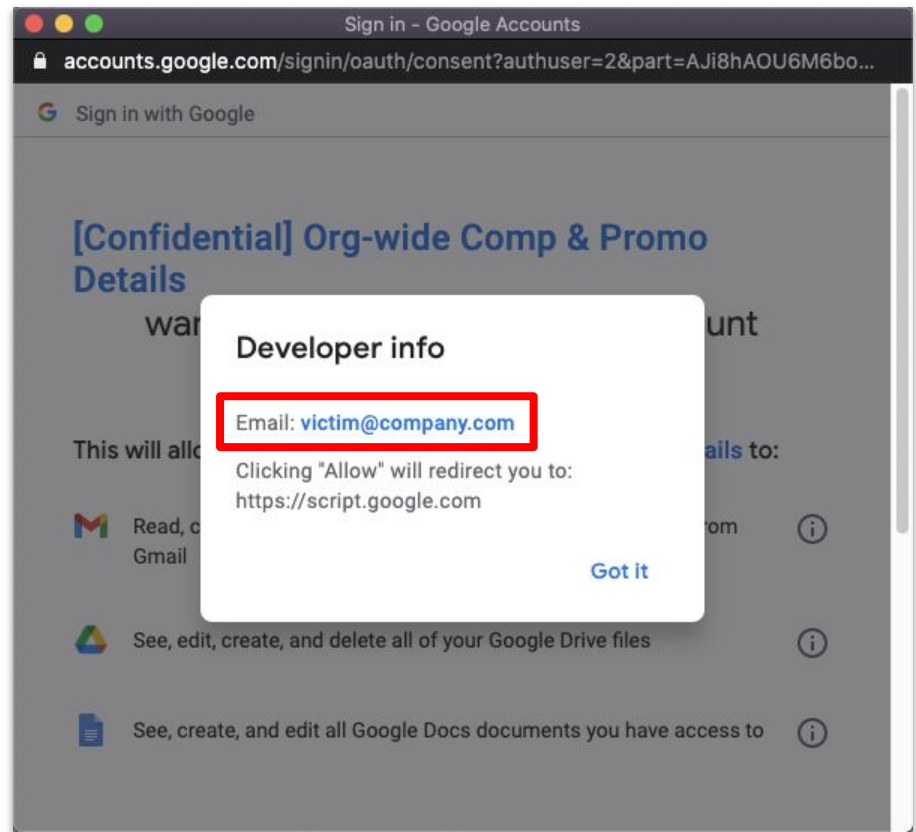
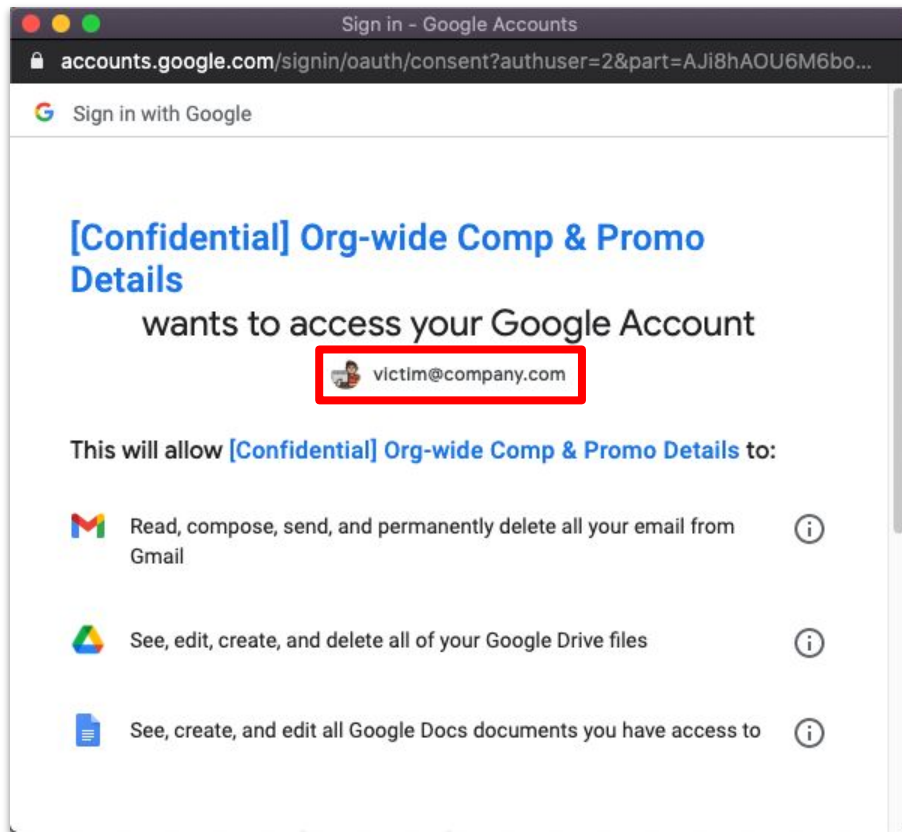


Victim
(In G Suite Org)

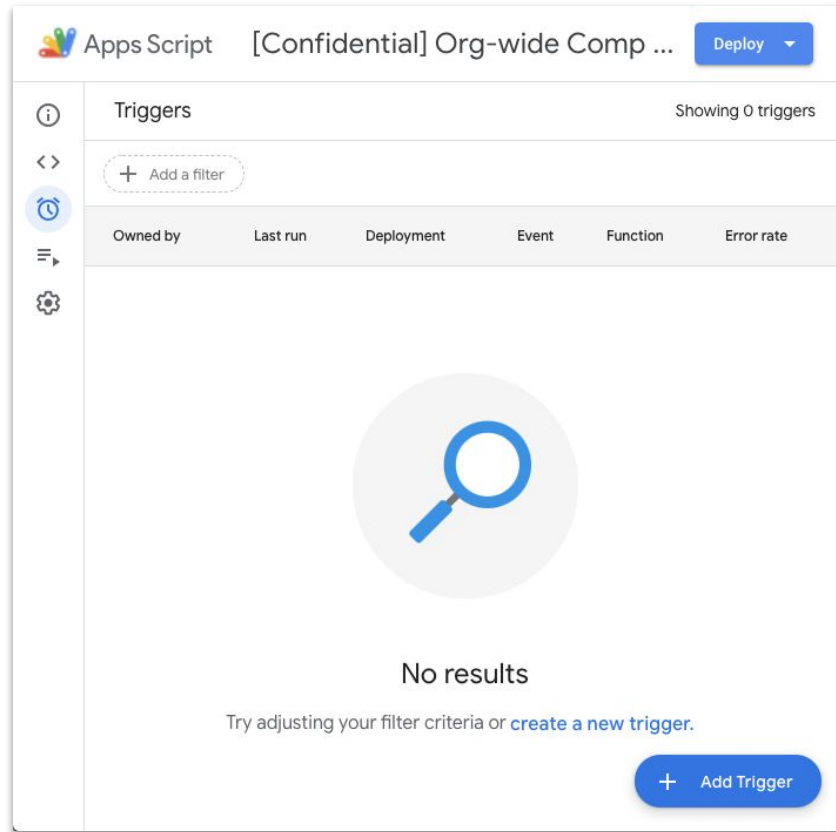
Victim activates trigger
and spawns prompt



Send copy URL to victim: **SUCCESS**



Not only does the victim become the owner of the Google Sheet, they also become the publisher of the attached App Script.



Problem: The Apps Script “Triggers” are not copied.
How will the victim trigger our payload?



Files



Execution log

appsscript.json

Code.gs

macros.gs

Libraries



Services



Gmail

```
1 {
2   "timeZone": "America/Los_Angeles",
3   "dependencies": {
4     "enabledAdvancedServices": [{
5       "userSymbol": "Gmail",
6       "serviceId": "gmail",
7       "version": "v1"
8     }]
9   },
10  "exceptionLogging": "STACKDRIVER",
11  "oauthScopes": ["https://mail.google.com/",
12                 "https://www.googleapis.com/auth/documents"],
13  "runtimeVersion": "V8",
14  "sheets": {
15    "macros": [{
16      "menuName": "Execute Macro",
17      "functionName": "ExecuteMacro"
18    }]
19  }
```



Files



Run



Debug

appsscript.json

Code.gs

macros.gs

Libraries



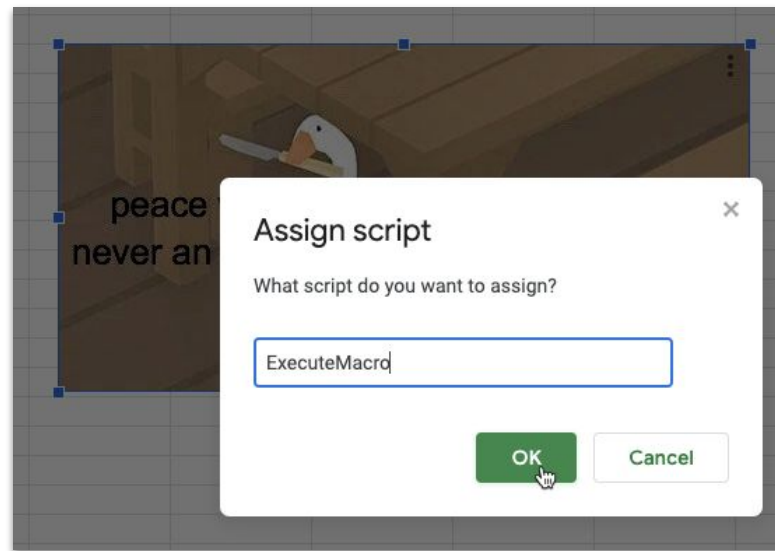
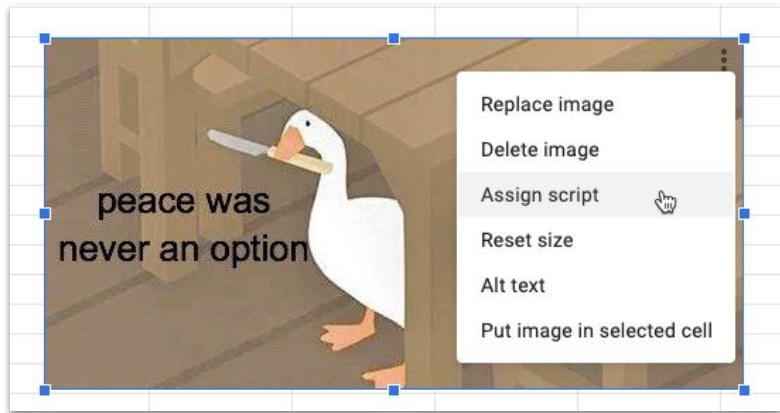
Services



Gmail

```
1 function ExecuteMacro()
2   // Your code here
3   Logger.info('Neat');
4 }
```

Example macro in Google Sheets



Assign a macro to execute upon image click in Sheets

BUT WAIT THERE'S MORE

BYPASSED

this message if a user tries to use an app that can't access restricted Google services

Message (500 characters limit)

Third-party OAuth app restricted by Google Workspace policy.

☒ Block all third-party API access

Requests by third-party apps are denied access to Google Workspace data and user data. This blocks all OAuth scopes, including sign-in scopes. [Learn more](#)

Another Tip for Defeating “Third-Party App”/Unverified App Restrictions

- A script “container” has the same Owner as the “container”.
- This means any Doc/Sheet/Slide which has an Owner inside a GSuite domain (e.g. company.com) is a bypass waiting to happen.
- If you have edit access to any Doc/Sheet/Slide made by someone in a GSuite domain, you can create an Apps Script app and bypass the “third-party app” restrictions!



All container-bound scripts use the same owner, viewer, and editor access list defined for the container file. The container owner takes ownership of a new script project regardless of who created it.

Post-Compromise Pivoting & Privilege Escalation

Moving right along

Pivoting to Google Cloud (GCP)

Accessing Google Cloud Through Apps Script

- If your Apps Script service has the `https://www.googleapis.com/auth/cloud-platform` scope, you can access any GCP API as the user who granted your app access.
- You can then use the access token from `ScriptApp.getOAuthToken()` in Authorization: `Bearer {{TOKEN}}`.

This will allow **Apps Script App** to:



See, edit, configure, and delete your Google Cloud Platform data



HTTP/1.1 403 Forbidden

Vary: Origin

Vary: X-Origin

Vary: Referer

Content-Type: application/json; charset=UTF-8

Date: Sun, 20 Jun 2021 03:20:24 GMT

Server: ESF

Cache-Control: private

X-XSS-Protection: 0

X-Frame-Options: SAMEORIGIN

X-Content-Type-Options: nosniff

Alt-Svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000,h3-T051=":443"; ma=2592000,h3-Q050=":443"; ma=2592000,h3-Q046=":443";

Connection: close

Content-Length: 1046

```
{
  "error": {
    "code": 403,
    "message": "Identity and Access Management (IAM) API has not been used in project 782839418626 before or it is disabled.",
    "status": "PERMISSION_DENIED",
    "details": [
      {
        "@type": "type.googleapis.com/google.rpc.Help",
        "links": [
          {
            "description": "Google developers console API activation",
            "url": "https://console.developers.google.com/apis/api/iam.googleapis.com/overview?project=782839418626"
          }
        ]
      }
    ]
  }
}
```

Some unknown project
number?



Some *mostly* undocumented fun...

- All Apps Scripts upon creation are allocated a hidden Google Project attached to them (yes really).
- This makes requests with your access token implicitly bound to this project, which has no APIs enabled.
- To get around this, you can specify the `x-goog-user-project` header followed by the project name you want to query.

GET /v1/projects/**PROJECT_ID**/serviceAccounts HTTP/1.1

Host: content-iam.googleapis.com

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:88.0) Gecko/20100101 Firefox/88.0

Accept: */*

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

x-goog-user-project: **PROJECT_ID**

Authorization: Bearer **ACCESS_TOKEN_FROM_APPS_SCRIPT**

Connection: close

ID of the GCP Project

`ScriptApp.getOAuthToken();`

Google Cloud Shell API

- Google Cloud Shell API allows you to add your public key, start, and connect to a virtual Linux instance via SSH.
- Using the API you can specify an access/ID token to authorize the pre-installed `gcloud` CLI for.

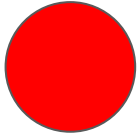


(docpusher-208120) × + ▾

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to docpusher-208120.
Use "gcloud config set project [PROJECT_ID]" to change to a different project.
mandatory@cloudshell:~ (docpusher-208120)$ ls
README-cloudshell.txt
mandatory@cloudshell:~ (docpusher-208120)$
```

Mining Google Drive for Paydirt

Sharing in Google Drive

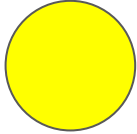


Get link

Restricted Only people added can open with this link

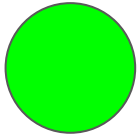
[Share with ExampleCompany](#)

[Copy link](#)



ExampleCompany ▾

Anyone in this group with this link can view



← Link settings



People in ExampleOrg can search for this file



Preset Defaults & Hardened Settings

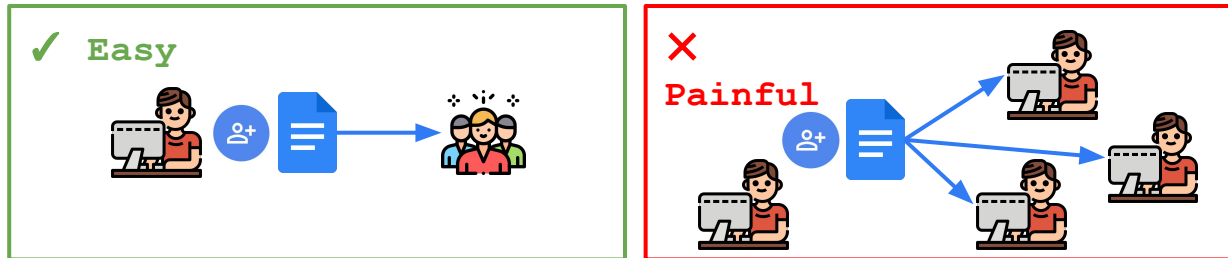
- By default sharing with the entire org is one-click away with the default being shared only with those explicitly added to the doc. This is the tightest default control available for link sharing.
- Once a G Suite org user views a document, it is searchable by them in the future.
- The document URLs are well outside the range of brute forcing.

None of your files or folders matched this search

Try another search, or click the arrow in the search box to find a file by type, owner and more [Learn more](#)

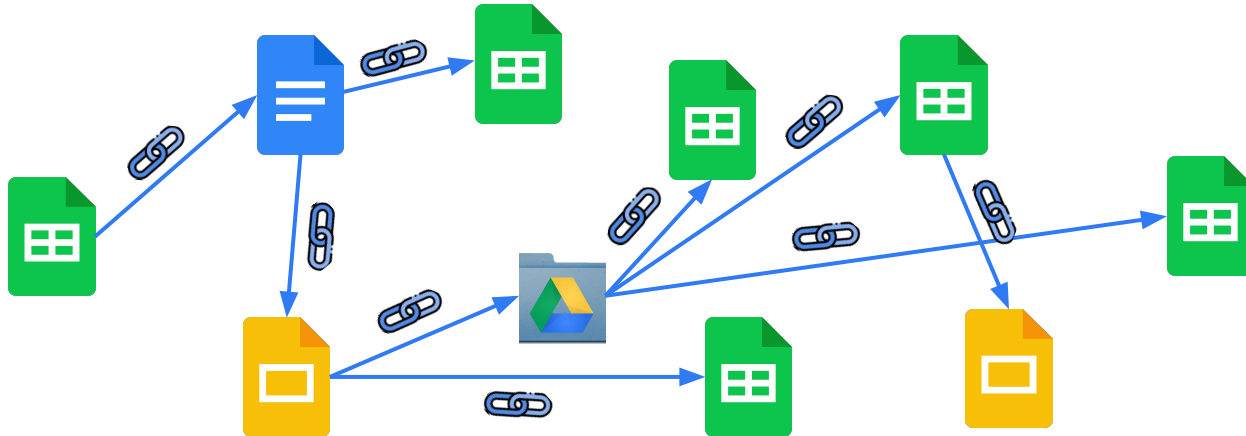
Real world usage: what *actually* happens?

- Any important file, by definition, is shared with other users.
- Adding individual users one-by-one to a doc is *tedious* (though Google Groups can be used).
- In practice, a large portion of the docs are just shared by link and only a tiny portion are searchable by the entire org.



How can we get access to these docs?

- Search internally-shared systems: chat, forums, Q&A sites, ticket management queues, bug trackers, etc.
- The same way we do on the web: spidering.



An Apps Script Spider

- Takes a list of seed Drive/Doc/Sheet/Slide links as starting points and recursively crawls using Apps Script until it's exhausted all paths.
- Along the way collects metadata for sharing, contents, creation, authors, etc for review.
- <https://github.com/mandatoryprogrammer/PaperChaser>

Dumping Employee Directory with the People API

Pull the Entire Organization's Directory via People API

- Requires the `https://www.googleapis.com/auth/directory.readonly` SCOPE
- Contains everything from names, emails, titles, phone numbers, birthdays, addresses, calendar URL, etc.
- This is very useful for further spear fishing, or for re-entry planning upon being detected and revoked.



People API

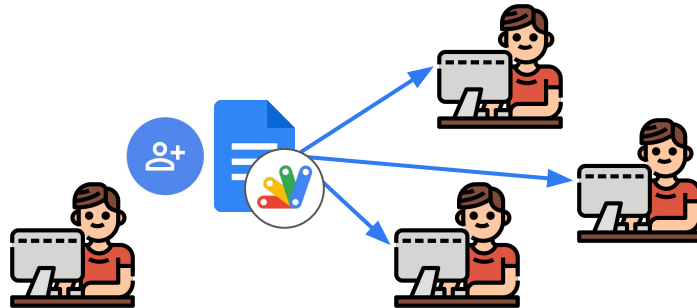
Privilege Escalation

The fastest way you'll move up in a company™

Exploiting Apps Script Attached to a Google Doc/Sheet/Slide

Revisiting Apps Script Bound to Docs/Sheets/Slides

- Recall we talked about Apps Script as being able to be “bound” to Docs/Sheets/Slides. The file a script is bound to is called the “container”.
- If that’s the case, does the Apps Script and the file it’s attached to have separate sharing permissions?



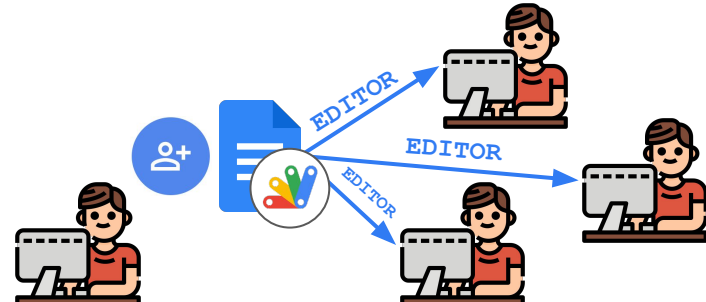


All container-bound scripts use the same owner, viewer, and editor access list defined for the container file. The container owner takes ownership of a new script project regardless of who created it.

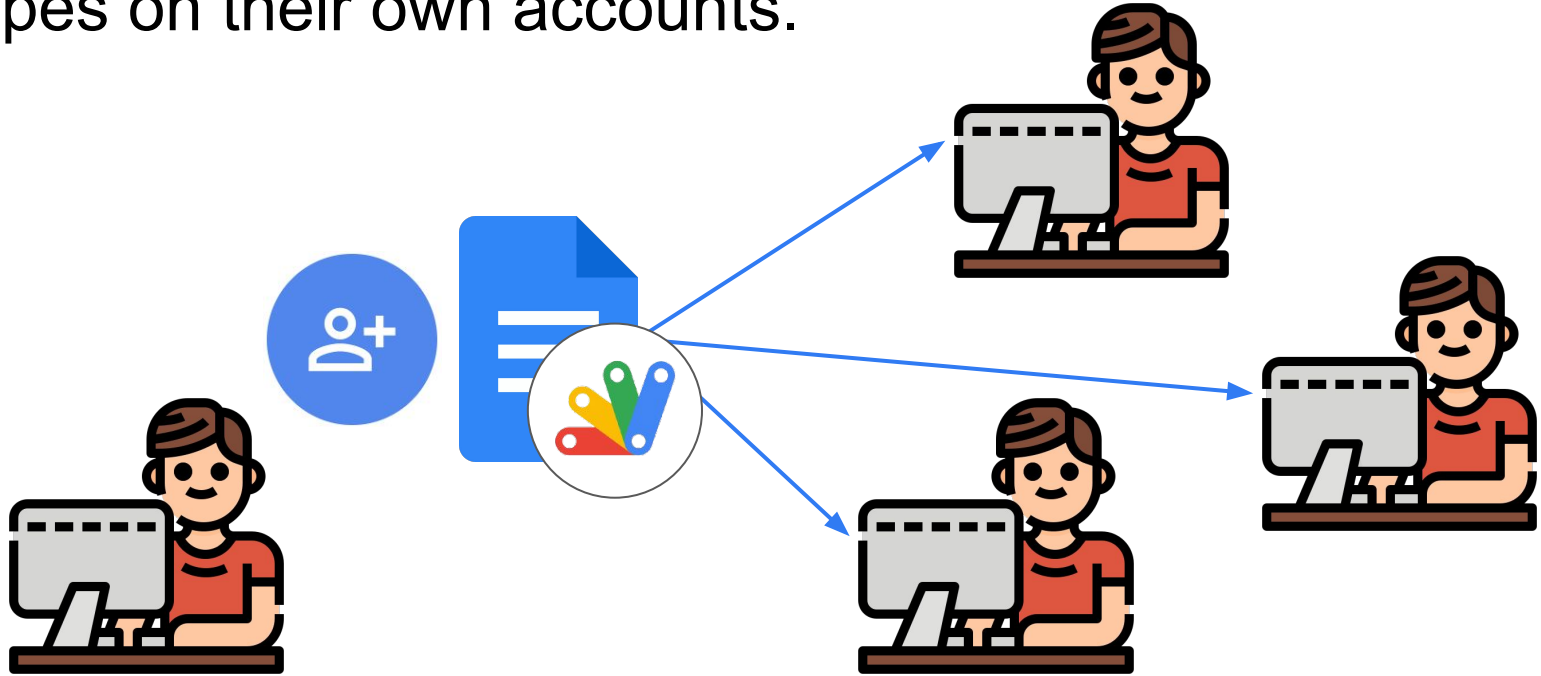
(<https://developers.google.com/apps-script/guides/bound>)

Another Interesting Question

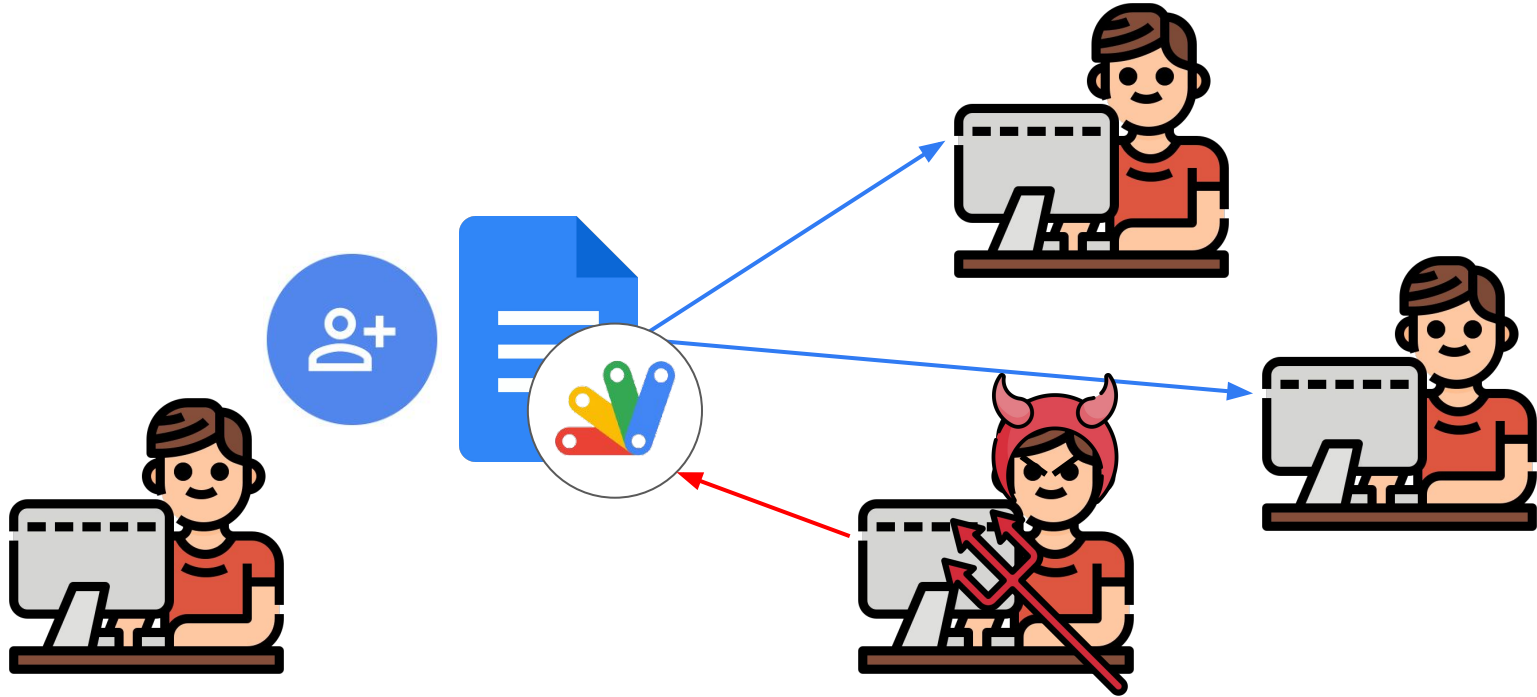
- Recall that “Edit” access is ***required*** to run the Apps Script.
- Whoever has “Edit” access to a given Doc/Sheet/Slide will also be able to Edit the Apps Script attached to it.
- What happens when you have a bunch of people all sharing the same Doc/Sheet/Slide which has Apps Script?



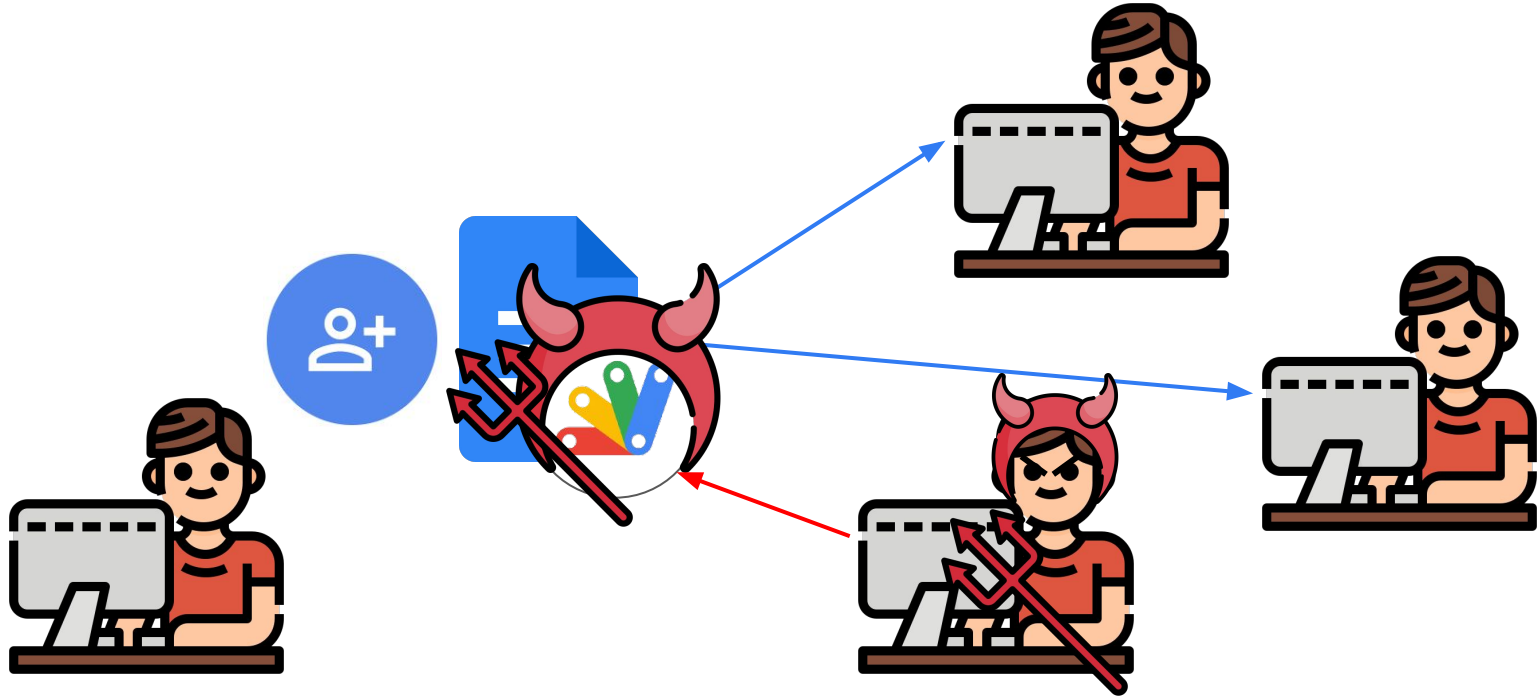
Situation: Many users have editor on a Doc With Apps Script. They have all granted the Apps Script OAuth scopes on their own accounts.



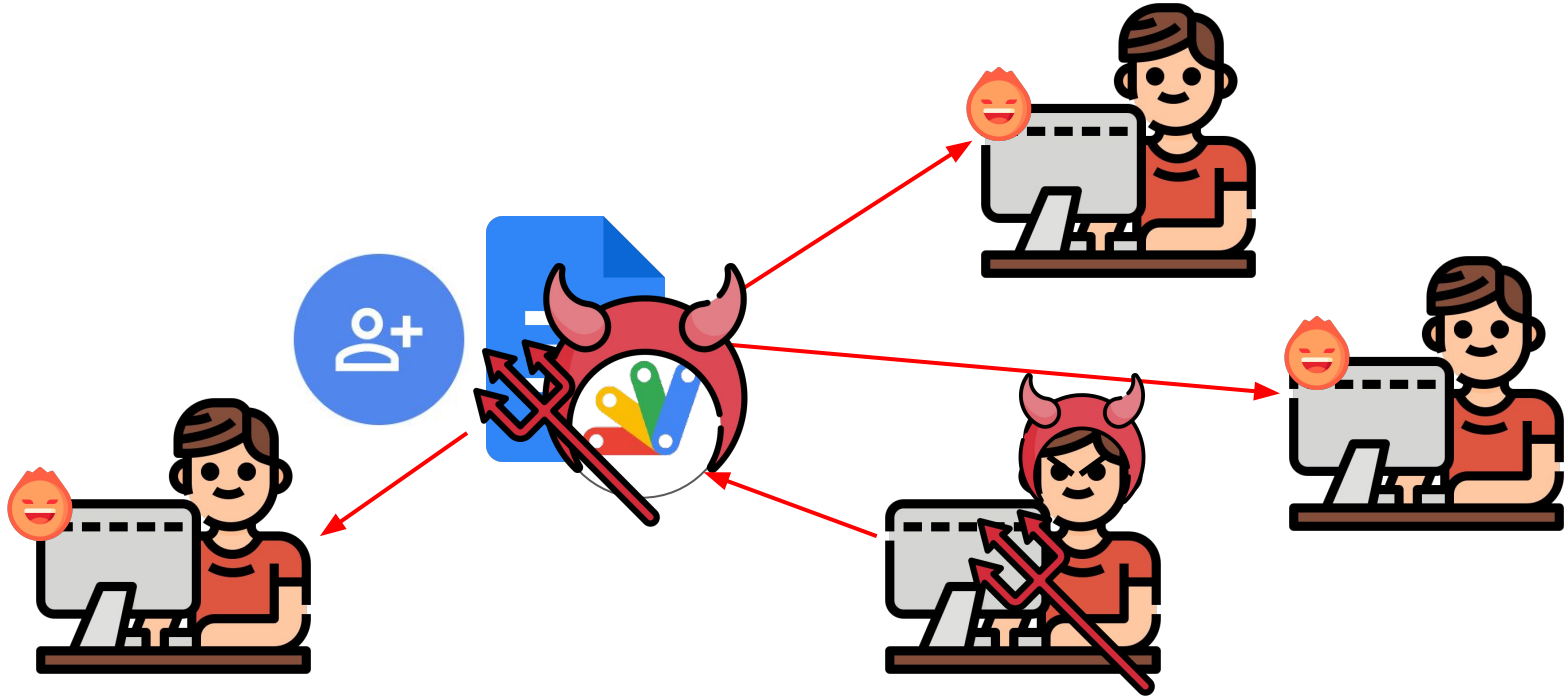
A malicious user has editor on the Doc as well.



The malicious user edits the Apps Script to contain a malicious script.



Upon each user triggering the already-authorized Apps Script, the malicious code runs as them.



Let's Do One Better

- This of course requires we wait for whatever triggers are in place to be re-fired.
 - In the case of background time-based triggers, that's fine we can wait.
- We can force a re-trigger by publishing the bound script as a web app. When this URL is visited by one of the users who authorized the script it will execute as them.
- This works even if triggered via `<img>`, etc!



Deploy ▼

New deployment

Manage deployments

Test deployments

New deployment

Select type



Configuration

Add-on

Web app

API Executable

New deployment

Select type



Configuration



Web app

Add-on

Description

New description

v1

Web app

Execute as

User accessing the web app



The web app will require users to authorize to run using their account data.

Who has access

Anyone with Google account



Add-on

This Apps Script project is using an Apps Script-managed Google Cloud Platform (GCP) project.

Cancel

Deploy

New deployment

Deployment successfully updated.

Version 2 on Jun 26, 2:59 PM

Deployment ID

AKfycbzk1asGh61fnVQqzoiJmTMHStsUHB87iA5QT-giusMIHdUQdlQrArmghm24sAQVSe_X

 [Copy](#)

Web app

URL

<https://script.google.com/macros/s/AKfycbzk1asGh61fnVQqzoiJmTMHStsUHB87iA5QT-giusMIHdUQdlQrArmghm24sAQ...>

 [Copy](#)

Done

Lateral Movement via Enumerating and Joining Open Google Groups

Google Groups, a PrivEsc Factory

- Google Groups are often used for ACL in GCP/IAM.
- By default, Google Groups are openly joinable by everyone in an org.

2 of 3 ✕

Choose privacy settings

Who can join group

Anyone in the organization can join ▼

Who can view conversations

☒ ☐ ☐ ☐

Group owners Group managers Group members Entire organization

Who can post

☒ ☐ ☐ ☐

Group owners Group managers Group members Entire organization

Who can view members

☒ ☐ ☐ ☐

Group owners Group managers Group members Entire organization

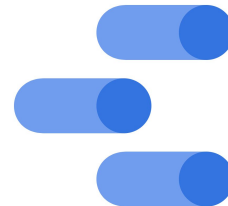
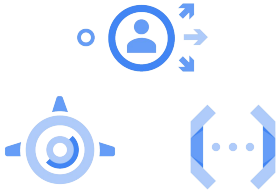
[Back](#) [Next](#)



What can be gated by Google Groups?



Google Cloud Platform



Google Group API Access via Apps Script

- Modifying Google Groups via Apps Script is not as easy as it sounds.
- The Google Groups Settings/Directory API is restricted to admins only.
- However, the “Cloud Identity API” is available to all users which allows some access to Google Groups via API.

What can you do via API for Google Groups?

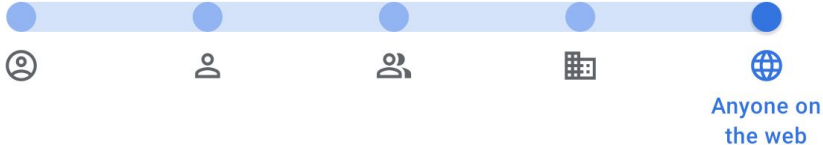
- List all Google Groups in a given organization.
- List Google Group's members and their roles.
- Create, update, delete, manage members of your own groups.
- **NOT** able to *join* an open Google Group via the API.

Better Spear Phishing With Google Groups

Google Group Trick: ***With Manual Access***

- Obtain any untaken email @company.com using Google Groups (external emails to Google Groups must be enabled for the org).
- Add and verify the email as an alias in Gmail and you can now send and receive mail at the address.

Who can post



Group owners can allow incoming email from outside the organization

Send mail as:

(Use OPSEC Mail Mail to send from your other email addresses)

[Learn more](#)

Jon Doe <jon@example.com>

[Add another email address](#)

Add another email address you own

Enter information about your other email address.

(your name and email address will be shown on mail you send)

Name:

Email address:

☒ Treat as an alias. [Learn more](#)

[Specify a different "reply-to" address](#) (optional)

Add another email address you own

Verify your email address

Before you can send mail as **finance@example.com**, we need to verify that you own this email address. To perform the verification click "Send Verification". We will then send an email to finance@example.com with instructions on how to verify your address.

OPSEC Mail Confirmation - Send Mail as finance@example.com

Inbox x



'OPSEC Mail Team' via finance <finance@example.com>

to finance ▼

You have requested to add finance@example.com to your OPSEC Mail account.

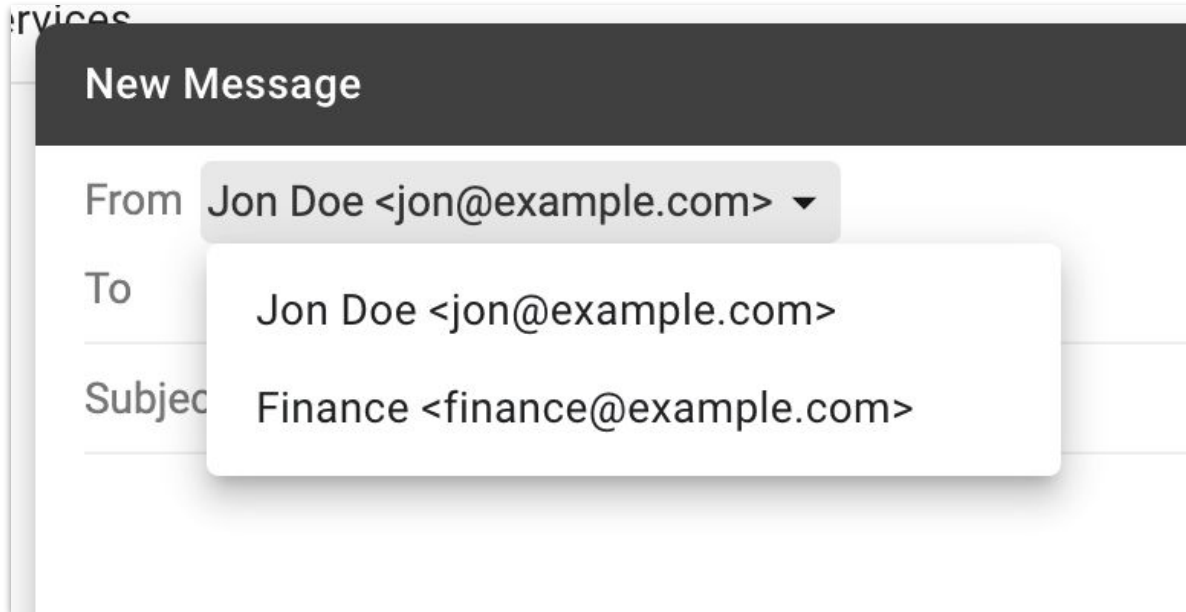
Confirmation code: 952838321

Before you can send mail from finance@example.com using your OPSEC Mail account (jon@example.com), please click the link below to confirm your request:

<https://mail.google.com/mail/f-%5BANGjdJ-gRdPikQu2-5XHdHluSwNxD3u1ZK7loBQwM5vpB4nH9q60iM--4dBcTzEVBaPbUehtE9IDSHI3COcDtBJklTU>

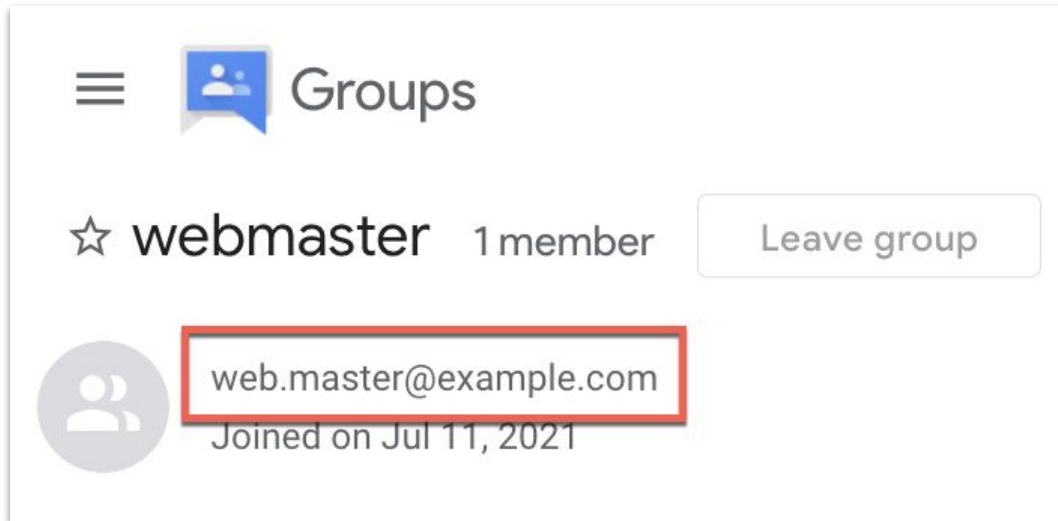
Google Group Trick: ***With Manual Access***

- You've now upgraded your spear-phishing game 200%.



Bonus Trick

- **Bonus:** If an alias is taken, you can add dots to it and receive emails at that address. This is (confusingly) the *opposite* behavior of regular Gmail which treats dots as irrelevant.



Backdoors, Stealth, & Persistence

Why not stick around?

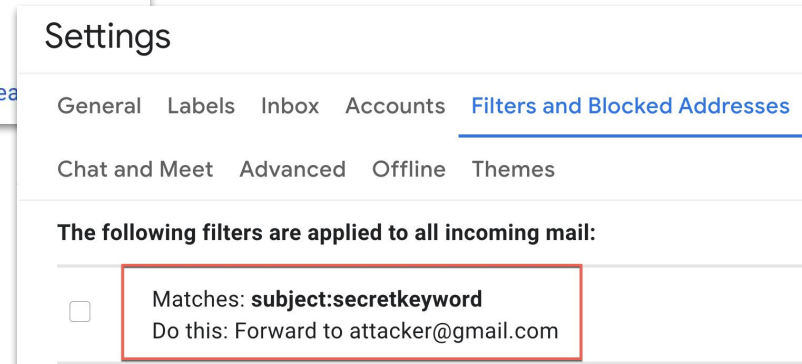
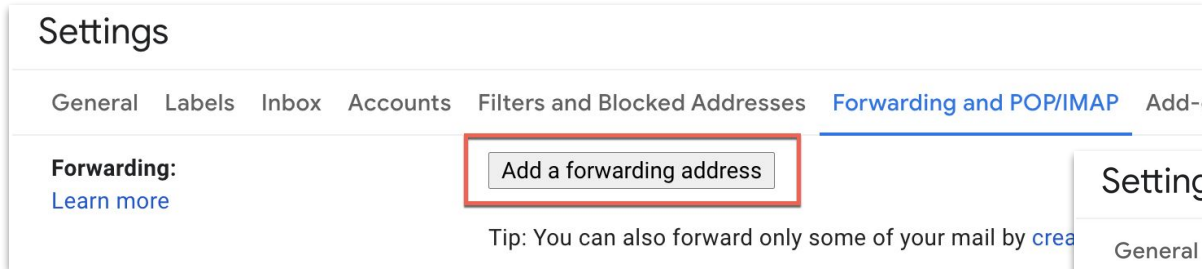
Gmail Trickery

Gmail Tricks: ***With API Access***

- Create a filter to hide security notifications from Google:
`from: (no-reply@accounts.google.com) "Security Alert"`
- Filter to hide password reset emails to be exfiltrated by Apps Script implant.
- Sadly creation of Delegates and Forwarding Addresses are intentionally restricted to service accounts with delegated domain-wide authority.

Gmail Tricks: *With Manual Access*

- Create forwarding address and create a filter to send matching emails to this address (or forward *all mail*).



Setting a More Deceptive App Name



Apps Script

Untitled project

Deploy



Files



Run



Debug

myFunction

Execution log

Use legacy editor



appsscript.json

Code.gs



Libraries



Services



```
1 function myFunction() {  
2  
3 }  
4
```

Homoglyph Attacks: Post Google Doc Worm

- After the Google Doc worm Google upgraded its anti-homoglyph prevention, banning similar-looking unicode characters, no-width-spaces, etc.
- However, the magic of the Right-to-Left Override (RTL) character (U+202E) still works.



Untitled project - Project Editor

script.google.com/u/2/home/projects/1LASZeIOLX10S9...

Update

Apps Script

Untitled project

Deploy

?

Files

+

appsscript.json

Code.gs

Libraries

Services

+

+

↶

↷

▶ Run

⌘ Debug

myFunction ▼

Execution log

Use legacy editor

```
1  function myFunction() {
2
3  }
4
```

Execution log

✕

10:52:16 PM

Warning

This project requires access to your Google Account to run. Please try again and allow it this time.

Perpetual Apps Script Execution

Keeping the Good Times Rolling

- Apps Script has time-based triggers allowing for background execution on a schedule.
- Can be run as often as every minute and executes as whatever user was running the script that created the trigger.

```
1  function someFunction() {  
2    // Run every minute  
3  }  
4  
5  function example() {  
6    // Create time-based trigger to execute every minute  
7    ScriptApp.newTrigger("someFunction").timeBased().everyMinutes(1).create();  
8  }  
9  
10
```

newTrigger(functionName)

Begins the process of creating an installable trigger that, when fired, calls a given function.

```
// Creates an edit trigger for a spreadsheet identified by ID.  
ScriptApp.newTrigger('myFunction')  
  .forSpreadsheet('1234567890abcdefghijklmnopqrstuvwxyz_a1b2c3')  
  .onEdit()  
  .create();
```

Parameters

Name	Type	Description
functionName	string	The function to call when the trigger fires. You can use functions from included libraries, such as <code>Library.libFunction1</code> .

Return

TriggerBuilder — An object used to continue the trigger-building process.

Authorization

Scripts that use this method require authorization with one or more of the following [scopes](#):

- `https://www.googleapis.com/auth/script.scriptapp`



Allow this application to run when you are not present



More of a Suggestion Really

- You can still create time triggers programmatically without declaring this scope.
- Time triggers can be created upon other triggers firing, they only require the user has authorized *some* scope to start firing.
- Persist indefinitely, no warnings required.



Backdoor Google Cloud Projects

Google Cloud Functions

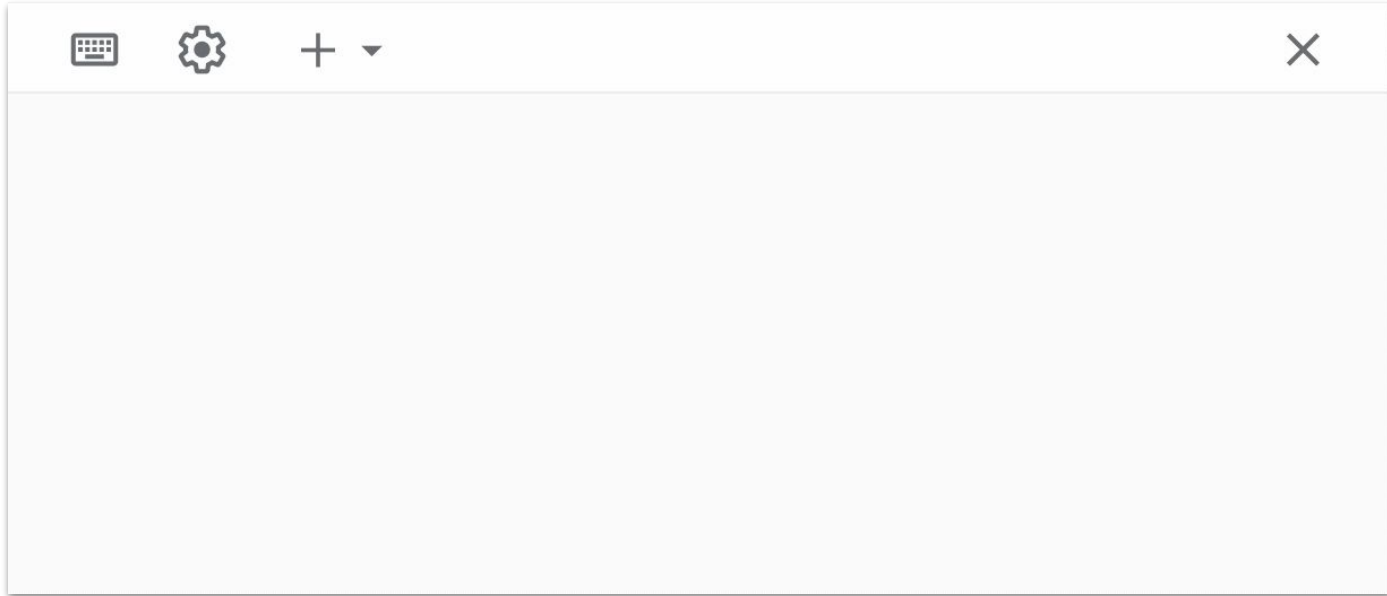
- Create Google Cloud Functions (GCFs) which execute as privileged service accounts on a project.

Editor	+ EDIT ROLE
iam.serviceAccounts.actAs	
iam.serviceAccounts.create	
iam.serviceAccounts.delete	
iam.serviceAccounts.disable	
iam.serviceAccounts.enable	
iam.serviceAccounts.get	
iam.serviceAccounts.getIamPolicy	
iam.serviceAccounts.list	
iam.serviceAccounts.update	

Owner	+ EDIT ROLE
iam.serviceAccounts.actAs	
iam.serviceAccounts.create	
iam.serviceAccounts.delete	
iam.serviceAccounts.disable	
iam.serviceAccounts.enable	
iam.serviceAccounts.get	
iam.serviceAccounts.getIamPolicy	
iam.serviceAccounts.list	
iam.serviceAccounts.setIamPolicy	
iam.serviceAccounts undelete	
iam.serviceAccounts.update	

Google Cloud Shell Backdoor

- Add a backdoor to the Google Cloud Shell `.bashrc` to exfiltrate access tokens or just connect back (Credit: [@89berner](#))



IAM Policy Backdoor

- Create IAM policies allowing your own projects access back in.

Add members to "App Engine default service account"

Add members and roles for "App Engine default service account" resource

Enter one or more members below. Then select a role for these members to grant them access to your resources. Multiple roles allowed. [Learn more](#)

New members

deployment-automation@prod-deploy-manager.iam.gserviceaccount.com



Role

Service Account Token Creator



Condition

[Add condition](#)



Impersonate service accounts (create OAuth2 access tokens, sign blobs or JWTs, etc).

[+ ADD ANOTHER ROLE](#)

☐

Send notification email

SAVE

CANCEL



That's all Folks!